Lesson 8 - Practical Application Guide

Implemented Validation Forms

1. Form Request: User Registration

Create:

```
php artisan make:request StoreUserRequest
```

Code:

```
class StoreUserRequest extends FormRequest
{
    public function authorize(): bool
    {
        return true;
    }

    public function rules(): array
    {
        return [
            'name' => 'required|string|max:255',
            'email' => 'required|email|unique:users,email',
            'password' => 'required|string|min:8|confirmed',
            'phone' => 'nullable|string|regex:/^[0-9]{10}$/',
            'birth_date' => 'required|date|before:today',
            'avatar' => 'nullable|image|mimes:jpeg,png,jpg|max:2048',
            'agree_terms' => 'required|accepted',
        ];
    }

    public function messages(): array
    {
        return [
            'name.required' => 'Name is required',
            'email.required' => 'Email is required',
            'email.unique' => 'Email already exists',
            'password.required' => 'Password is required',
            'password.min' => 'Password must be at least 8 characters',
            'password.confirmed' => 'Password confirmation does not match',
            'agree_terms.accepted' => 'You must agree to terms',
        ];
```

```
    }
}
```

Usage in Controller:

```
public function store(StoreUserRequest $request)
{
    $validated = $request->validated();

    if ($request->hasFile('avatar')) {
        $validated['avatar'] = $request->file('avatar')->store('avatars',
'public');
    }

    $validated['password'] = Hash::make($validated['password']);

    User::create($validated);

    return redirect()->route('users.index')->with('success', 'Registered
successfully');
}
```

---

2. Form Request: Create Post

```
class StorePostRequest extends FormRequest
{
    public function authorize(): bool
    {
        return auth()->check();
    }

    public function rules(): array
    {
        return [
            'title' => 'required|string|max:255',
            'slug' => 'required|string|unique:posts,slug|max:255',
            'content' => 'required|string|min:100',
            'excerpt' => 'nullable|string|max:500',
            'category_id' => 'required|exists:categories,id',
            'tags' => 'nullable|array|max:5',
            'tags.*' => 'exists:tags,id',
            'featured_image' => 'nullable|image|max:2048',
            'status' => 'required|in:draft,published',
```

```
            'published_at' =>
'required_if:status,published|nullable|date|after_or_equal:today',
        ];
    }
}
```

---

3. Custom Rule: Strong Password

Create:

```
php artisan make:rule StrongPassword
```

Code:

```
class StrongPassword implements ValidationRule
{
    public function validate(string $attribute, mixed $value, Closure $fail): void
    {
        if (!preg_match('/[A-Z]/', $value)) {
            $fail('Password must contain at least one uppercase letter');
            return;
        }

        if (!preg_match('/[a-z]/', $value)) {
            $fail('Password must contain at least one lowercase letter');
            return;
        }

        if (!preg_match('/[0-9]/', $value)) {
            $fail('Password must contain at least one number');
            return;
        }

        if (!preg_match('/[@$!%*?&]/', $value)) {
            $fail('Password must contain at least one special character');
        }
    }
}
```

Usage:

```
use App\Rules\StrongPassword;

$request->validate([
    'password' => ['required', 'min:8', new StrongPassword],
]);
```

---

4. Form Request: Order

```
class StoreOrderRequest extends FormRequest
{
    public function rules(): array
    {
        return [
            'items' => 'required|array|min:1',
            'items.*.product_id' => 'required|exists:products,id',
            'items.*.quantity' => 'required|integer|min:1',

            'payment_method' => 'required|in:cash,card,transfer',
            'card_number' => 'required_if:payment_method,card|digits:16',
            'card_cvv' => 'required_if:payment_method,card|digits:3',

            'shipping_address' => 'required|string|max:500',
            'shipping_city' => 'required|string|max:100',
            'shipping_zip' => 'required|digits:5',
            'notes' => 'nullable|string|max:1000',
        ];
    }
}
```

Blade Examples

Form with Validation Errors

```
<form action="{{ route('posts.store') }}" method="POST">
    @csrf

    {{-- Show all errors --}}
    @if ($errors->any())
```

```
        <div class="alert alert-danger">
            <ul>
                @foreach ($errors->all() as $error)
                    <li>{{ $error }}</li>
                @endforeach
            </ul>
        </div>
    @endif

    {{-- Title field --}}
    <div class="form-group">
        <label for="title">Title</label>
        <input
            type="text"
            name="title"
            id="title"
            value="{{ old('title') }}"
            class="form-control @error('title') is-invalid @enderror"
        >
        @error('title')
            <span class="invalid-feedback">{{ $message }}</span>
        @enderror
    </div>

    {{-- Content field --}}
    <div class="form-group">
        <label for="content">Content</label>
        <textarea
            name="content"
            id="content"
            class="form-control @error('content') is-invalid @enderror"
        >{{ old('content') }}</textarea>
        @error('content')
            <span class="invalid-feedback">{{ $message }}</span>
        @enderror
    </div>

    <button type="submit" class="btn btn-primary">Save</button>
</form>
```

Various Validation Examples

1. Controller Validation

```
public function store(Request $request)
{
    $validated = $request->validate([
        'title' => 'required|max:255',
        'content' => 'required',
    ], [
        'title.required' => 'Title is required',
        'title.max' => 'Title must not exceed 255 characters',
    ]);

    Post::create($validated);
}
```

2. Array Validation

```
$request->validate([
    'products' => 'required|array|min:1',
    'products.*.name' => 'required|string|max:255',
    'products.*.price' => 'required|numeric|min:0',
    'products.*.quantity' => 'required|integer|min:1',
]);
```

3. Conditional Validation

```
public function rules(): array
{
    $rules = [
        'title' => 'required|string|max:255',
    ];

    if ($this->input('type') === 'video') {
        $rules['video_url'] = 'required|url';
        $rules['duration'] = 'required|integer';
    }

    return $rules;
}
```

Useful Commands

```
# Create Form Request
php artisan make:request StorePostRequest
php artisan make:request UpdatePostRequest

# Create Custom Rule
php artisan make:rule Uppercase
php artisan make:rule StrongPassword
```

Best Practices

1. Use Form Requests for complex validation
2. Use validated() instead of all()
3. Write clear error messages
4. Use old() to preserve values
5. Use Custom Rules for complex logic
6. Check authorization() in Form Request

Next Step

Lesson 9: File Upload and Storage

Happy Learning!